

Vertos Archive: AI-Powered Academic Platform

Comprehensive Project Report

1. Introduction

Vertos Archive is a full-stack, AI-powered academic resource platform built for students at Lovely Professional University (LPU). It allows students to upload, search, and chat with course materials including notes, previous year question papers (PYQs), syllabi, placement resources, and more, powered by a Hybrid RAG (Retrieval-Augmented Generation) engine.

2. Core Features

The platform encompasses several core modules designed for optimal user experience and scalable operations.

- **Document Upload & Processing:**
 - Supports 6 formats: PDF, DOCX, PPTX, Images, and scanned documents.
 - Multi-image upload to group multiple image pages under one document.
 - Automatic text extraction via pdf-parse, officeParser, and GPT-4o Vision API.
 - Smart OCR Fallback: Automatically re-processes sparse/handwritten PDFs with Vision OCR.
 - **Admin Moderation Queue:**
 - Review, approve, or reject student uploads.
 - Edit metadata (title, subject, category) to fix user errors.
 - Duplicate detection using title/subject similarity search.
 - Constructive feedback system via review notes.
 - **Gamification & Leaderboard:**
 - Contributors earn points (+10 per approved upload) and trust scores.
 - Automatic badge progression: Top Contributor (50pts), Elite Verto (100pts).
 - Live leaderboard ranking and global community stats.
 - **Homepage Intelligence:**
 - Dynamic Popular Searches based on platform analytics.
 - Trending Topics derived from active subjects.
 - Community Uploads Feed and AI-generated Suggested Questions.
 - **User Accounts & Security:**
 - JWT-based email/password registration and Google OAuth.
 - Enterprise-Grade Security: Anti-prompt injection (payload capping, sanitization).
 - Global API rate limiting (10 req/15m for auth).
 - Recursive XSS cleaning and NoSQL injection sanitization.
-

3. RAG Features (Retrieval-Augmented Generation)

The heart of Vertos Archive is **Verto AI**, an intelligent chatbot that answers student questions based on the uploaded materials.

- **Hybrid Search Engine (RRF):**
 - Combines **Semantic Vector Search** (Qdrant) and **Keyword Search** (MongoDB \$text).
 - Uses **Reciprocal Rank Fusion (RRF)** to merge results, ensuring both semantic meaning and exact matches (e.g., specific course codes or years) are retrieved accurately.
- **Smart Exam Generation:** Context-aware generation of CA, Mid-Term, ETE, and ETP papers strictly aligned with syllabus topics and PYQ patterns.
- **Streaming Responses (SSE):** Utilizes Server-Sent Events for real-time token-by-token output, ensuring a fast, interactive chat experience similar to ChatGPT.

- **Source Citations:** Every answer links back to the exact chunks and documents used to generate it, ensuring high trust and verifiability.

- **Contextual Filtering:** Chat can be filtered by specific categories (Notes, Syllabus, PYQs, Placements, etc.) or subjects.

4. System Architecture

The application follows a modern decoupled architecture.

- **Frontend (Client):** Built with React 18 and Vite. Handles UI/UX, state management (Context API), and real-time streaming displays. Deployed on Vercel.
- **Backend (Server):** Node.js and Express.js providing REST APIs. Handles auth, database interactions, and orchestrates the AI pipeline. Deployed on Render.
- **Primary Database:** MongoDB Atlas stores structured data (users, document metadata, chat history, analytics).
- **Vector Database:** Qdrant Cloud stores 1536-dimensional embeddings (via OpenAI text-embedding-3-small) for fast Approximate Nearest Neighbor (ANN) search.
- **Storage & AI:** Cloudinary handles raw file storage. OpenAI API powers embeddings and GPT-4o-mini generation.

AI Processing Pipeline (Background Job)

1. **Chunking:** Raw text is split into 1000-character chunks with 200-character overlaps.
 2. **Embedding:** Chunks are sent to OpenAI to generate vector embeddings.
 3. **Indexing:** Vectors + metadata are pushed to Qdrant; metadata is indexed in MongoDB.
 4. **Insights:** GPT-4o-mini generates key topics and suggested questions dynamically.
-

5. DFD & Flow Diagrams

Level 0 DFD (Context Diagram)

- **Student** -> (Uploads Document / Asks Question) -> **Vertos Archive System** -> (Returns Answers / Citations / Leaderboard Stats)
- **Admin** -> (Approves/Rejects Documents) -> **Vertos Archive System**
- **Vertos Archive System** -> (Interacts with) -> **Cloudinary / MongoDB / Qdrant / OpenAI**

Data Flow - Upload Pipeline

1. **Student** submits file.
2. File saved to **Cloudinary**; URL saved to **MongoDB** (PendingDocument).
3. Text extracted via Parsers/OCR and saved.
4. **Admin** reviews and approves.
5. **Pipeline Worker** chunks text -> generates Embeddings (OpenAI) -> saves to **Qdrant** and **MongoDB**.

Data Flow - Query Pipeline (Hybrid RAG)

1. **Student** asks a question.
 2. Question is vectorized via **OpenAI**.
 3. **Qdrant** performs Vector Search (Top 60).
 4. **MongoDB** performs Keyword Search (Top 10).
 5. Results merged via **RRF Fusion**.
 6. Merged context sent to **GPT-4o-mini**.
 7. AI streams answer via **SSE** back to Student.
-

Report auto-generated by Verto AI / Antigravity System